VEO Technical Documentation

FULL STACK AI-POWERED SOURCING APPLICATION

Frontend — Next.js

1. Project Structure

- **Framework:** Next.js (TypeScript)
- **Entry Point:** app/page.tsx
- **Global Layout:** Defined in app/layout.tsx with custom fonts and metadata.
- **UI Components:** Located in components/ui/ (e.g., carousel.tsx, scroll-area.tsx, progress.tsx, chart.tsx, sidebar.tsx)
- **Feature Components:** E.g., components/job-detail-view.tsx, components/jobs-dashboard.tsx — handle dashboard views and job/candidate details.

2. Configuration

- next.config.ts: Central place for Next.js config options.
- **Fonts:** Uses next/font for automatic optimization and custom Vercel Geist fonts.

3. API Integration

- **API Routes:** Located in app/api/
  - **Jobs API:** app/api/jobs/route.ts — Handles job listing, extraction of requirements, department assignment, and applicant priority.
  - **Skills Extraction:** app/api/skills-from-cv/route.ts — Receives CV uploads, passes data to FastAPI backend, and returns extracted skills.
  - **Display Skills:** app/api/display-skills/route.ts — Fetches all skills from backend.

- **CV Serving:** app/api/cv/[...path]/route.ts`— Serves CV files with correct content-type headers.
- **Backend Communication:** Uses `fetch` to communicate with Python/FastAPI backend. Backend URL is set via `process.env.BACKEND_URL`.

4. UI Patterns

- **Dashboard:** `components/jobs-dashboard.tsx` — Shows jobs, candidates, paginates, includes filters and batch operations.
- **Job Detail Views:** `components/job-detail-view.tsx` — Dynamic import for PDF viewer, paginated candidates, detailed scoring.
- **Carousel Pattern:** `components/ui/carousel.tsx` — Uses Embla Carousel for horizontal/vertical scrollable content.
- **Progress/Chart:** UI feedback via progress bars and charts, powered by Radix UI and Recharts.

5. Styling

- **CSS:** Global styles in `app/globals.css`. Component-level styles via Tailwind utility classes.
- **Theme:** Light/Dark themes supported in chart components.

6. Deployment

Local Dev: `npm run dev` or equivalent.

Backend — Python/FastAPI

1. Project Structure

- **Main App**: `src/resume/main.py`
- **Skill Extraction Logic:** `src/resume/skill_extraction.py`
- **Custom Tools:** `src/resume/tools/custom_tool.py`
- **Database Layer:** Uses SQLite (`candidate_reports.db`), SQLAlchemy ORM (models, session management).

2. FastAPI Endpoints

CORS: Enabled for frontend integration.

- **Skill Extraction:**
  - `/extract-skills-from-cv` — Accepts PDF or job description, extracts skills via Azure AI, saves results to DB, supports both candidates and jobs.
  - `/extract-skills` — Uses Azure AI to extract and categorize skills from job description.
  - `/display_skills` — Returns all candidates and their extracted skills.
- **Job & Candidate Management:**
  - `/barem` — Creates or updates scoring criteria ("barem") for jobs, stores in DB.
  - `/job-barem/{job_title}` — Gets barem for a given job.
  - `/job-skills/{job_title}` — Gets required skills for a job.
  - `/job-recommended-candidates/{job_title}` — Returns top candidates for a job.
- **Analysis:**
  - `/analyze` — Analyzes candidate CVs against job descriptions and barem criteria, returns detailed report, stores results in DB.
  - `/reports/{report_id}` [DELETE] — Deletes a report.
  - `/stats` — Global DB stats.
  - `/job-positions`, `/candidates/search` — For dashboards and filtering.

3. Skill Extraction Logic

- **Azure AI Integration**: Used for natural language skill extraction from job descriptions and CVs.
- **Custom PDF Tool:** Extracts content and skill proficiency from PDFs with support for both visual (progress bars, star ratings) and text-based indicators.
- **Categories:** Skills are grouped into standardized categories (programming, data, BI, web, cloud/devops, ML/AI, mobile, etc.).

4. Database

- **Tables:**
  - `job_required_skills`: job title, required skills JSON, barem JSON.

- `extracted_skills`: candidate name, skill data.
- `candidate_reports`: stores analysis results, scores, strengths/gaps, etc.
- `Operations`:Insert/update on skill extraction and barem creation; reports updated or created as candidates apply/analyze.

5. Error Handling & Logging

- **Exceptions:** Handled, logged, and error messages returned in JSON.
- **Validation:** Ensures only one of PDF or job description is submitted; job title required for job description.

6. Integration with Frontend

- **Endpoints:** All major endpoints are consumed by Next.js frontend via API routes.
- **Return Format:** JSON responses for all endpoints, with structured data for rendering dashboards, charts, and job/candidate views.

How to Use

1. Installation

```bash
git clone -b finale https://srv-devcenteras-01.noz.local/tu/sourcing-ai.git
cd VEO-Next-Js
```

2. Backend Setup

```bash
cd backend
python3 -m venv venv
# On Windows:
venv\Scripts\activate
```

```
# On Mac/Linux:
source venv/bin/activate
pip install -r requirements.txt
```

Create a .env file in the backend directory with the following structure (replace with your own values):

```
AZURE_AI_API_KEY=your_api_key
AZURE_AI_ENDPOINT=https://your-endpoint
AZURE_AI_API_VERSION=your_version
model=azure/Llama-4-Maverick-17B-128E-Instruct-FP8
RECRUITEE_API_TOKEN=your_recruitee_token
```

3. Run Backend Server

```bash
python -m uvicorn src.resume.main:app --reload --port 8000
```

4. Frontend Setup

```bash
cd ../frontend
npm install
```

5. Run Frontend Server

```bash
npm run dev
```

or

```
yarn dev
```

```

Navigate to [http://localhost:3000](http://localhost:3000) in your browser.

## Example Workflow

- **Data Preparation:**

All candidate CVs (PDF files) and job folders (with job descriptions) are placed in the `assets` directory.

Note: This mimics future integration with an external recruitment system such as Recruitee, where CVs and job data will be sourced automatically.

- **Skill Extraction:**

The system extracts skills for all candidates (from their CVs) and all jobs (from their descriptions) and stores this data in the database. This initial extraction is necessary for recommendations and analysis.

- **Job Selection & Recommendations:**

When you click on any job in the dashboard, the recommendation system activates.

  - It recommends candidates who **did not apply** for this job but whose skills match the requirements and have potential for acceptance.
  - Candidates who have applied are also listed, but recommendations highlight new potential matches.

- **Barem Check:**

If the selected job does not have an assessment criteria configured, the system notifies you and prompts you to set it up. The assessment criteria is used for scoring and analysis.
```

- **Candidate Analysis:**

You can analyze all candidates for the selected job—those who applied **and** those recommended by the system.

  - The analysis uses the barem and extracted skills to generate scores, strengths/gaps, and fit reports.

Note: Both projects require proper folder structure and environment configuration for full integration. Backend must be running before using frontend features.
